

#MY SQL IMPLEMENTATION QUERIES

#1. SIMPLE QUERY

#Query: List all events happening after 2024 in Back Bay Area

```
SELECT event_id, title, type, location, event_date
FROM Event
WHERE location LIKE '%Back Bay%'
AND event_date >= '2024-01-01'
ORDER BY event_date;
```

#2. AGGREGATE

#Query: Full Funding Statistics Per Startup

```
SELECT
s.startup_id,
s.name AS startup_name,
SUM(fr.amount) AS total_funding,
COUNT(fr.funding_id) AS num_rounds,
AVG(fr.amount) AS avg_round_amount,
MIN(fr.amount) AS min_round_amount,
MAX(fr.amount) AS max_round_amount
FROM Startup s
LEFT JOIN FundingRound fr
ON fr.startup_id = s.startup_id
GROUP BY s.startup_id, s.name
ORDER BY total_funding DESC;
```

#3. INNER JOIN

#Query: List mentors with their matched startups

```
SELECT p.name AS mentor_name, s.name AS startup_name, sm.role
FROM Mentor m
```

```
JOIN People p ON p.person_id = m.person_id
JOIN StartupMentor sm ON sm.mentor_id = m.mentor_id
JOIN Startup s ON s.startup_id = sm.startup_id
ORDER BY mentor_name;
```

#4. NESTED QUERY (Subquery in WHERE clause)

#Query: Entrepreneurs who founded more than one startup

```
SELECT e.entrepreneur_id, p.name
FROM Entrepreneur e
JOIN People p ON p.person_id = e.person_id
WHERE e.entrepreneur_id IN (
    SELECT entrepreneur_id
    FROM EntrepreneurStartup
    GROUP BY entrepreneur_id
    HAVING COUNT(startup_id) > 1
);
```

#5. SUBQUERY IN FROM (Derived Table)

#Query: Average funding per industry (industry-level performance)

```
SELECT industry, AVG(total_funding) AS avg_industry_funding
FROM (
    SELECT s.industry, COALESCE(SUM(fr.amount),0) AS total_funding
    FROM Startup s
    LEFT JOIN FundingRound fr ON fr.startup_id = s.startup_id
    GROUP BY s.startup_id, s.industry
) AS industry_funding
GROUP BY industry
ORDER BY avg_industry_funding DESC;
```

#6. SUBQUERY IN SELECT

#Query: How many startups each entrepreneur has founded

```
SELECT
    p.name AS entrepreneur_name,
    e.entrepreneur_id,
    (SELECT COUNT(*)
     FROM EntrepreneurStartup es
     WHERE es.entrepreneur_id = e.entrepreneur_id
    ) AS total_startups
FROM Entrepreneur e
JOIN People p ON p.person_id = e.person_id;
```

#7. CORRELATED SUBQUERY

#Query: Events with above-average participation compared to their own event type

```
SELECT e.event_id, e.title, e.type,
    (SELECT COUNT(*)
     FROM EntrepreneurEvent ee
     WHERE ee.event_id = e.event_id) AS attendees
FROM Event e
WHERE (SELECT COUNT(*)
       FROM EntrepreneurEvent ee
       WHERE ee.event_id = e.event_id) >
    (SELECT AVG(x.attendee_count)
     FROM (
         SELECT event_id, COUNT(*) AS attendee_count
         FROM EntrepreneurEvent
         GROUP BY event_id
     ) x);
```

#8. ALL

#Query: Find startups whose total funding raised is greater than ALL investors' minimum investment capacity.

```
SELECT
```

```

s.startup_id,
s.name AS startup_name,
SUM(fr.amount) AS total_funding
FROM Startup s
JOIN FundingRound fr ON fr.startup_id = s.startup_id
GROUP BY s.startup_id, s.name
HAVING SUM(fr.amount) > ALL (
    SELECT capacity_min
    FROM Investor
);

```

#9. ANY

#Query: Find investors whose minimum investment capacity is greater than the smallest funding amount of ANY startup.

```

SELECT
    i.investor_id,
    p.name AS investor_name,
    i.capacity_min
FROM Investor i
JOIN People p ON p.person_id = i.person_id
WHERE i.capacity_min > ANY (
    SELECT amount
    FROM FundingRound
    WHERE amount IS NOT NULL
);

```

#10. EXISTS + NOT EXISTS

#Query: Find entrepreneurs who have founded at least one startup AND have never attended any event.

```

SELECT
    e.entrepreneur_id,
    p.name AS entrepreneur_name

```

```

FROM Entrepreneur e
JOIN People p ON p.person_id = e.person_id
WHERE EXISTS (
    SELECT 1
    FROM EntrepreneurStartup es
    WHERE es.entrepreneur_id = e.entrepreneur_id
)
AND NOT EXISTS (
    SELECT 1
    FROM EntrepreneurEvent ee
    WHERE ee.entrepreneur_id = e.entrepreneur_id
);

```

#11. UNION

#Query: All people who attended events as entrepreneurs or mentors

```

SELECT ep.event_id, ep.entrepreneur_id AS participant_id, 'Entrepreneur' AS role
FROM EntrepreneurEvent ep
UNION
SELECT mp.event_id, mp.mentor_id AS participant_id, 'Mentor' AS role
FROM MentorEvent mp;

```

#12. LEFT OUTER JOIN

#Query: Show all startups and any funding rounds they have (including startups with no funding)

```

SELECT
    s.startup_id,
    s.name AS startup_name,
    fr.funding_id,
    fr.amount
FROM Startup s
LEFT JOIN FundingRound fr
ON s.startup_id = fr.startup_id

```

```
ORDER BY s.startup_id;  
GROUP BY s.startup_id, s.name  
ORDER BY total_funding DESC;
```